

Project Planning

Project: **FitTrack - AI** – Personal AI-powered fitness tracker & health recommendations

This phase converts the identified requirements and design into a practical development sequence. The plan follows the actual modules present in the supplied repository.

1. Development Work Breakdown

STEP	PLANNED WORK	OUTPUT
1	Project setup and dependency installation	Node.js project, <code>package.json</code> and required dependencies.
2	Environment configuration	<code>PORT</code> , <code>MongoDB URI</code> , <code>JWT secret</code> , <code>Fitness API key</code> , and <code>Gemini key</code> configuration.
3	Database layer	MongoDB connection plus User, Workout, and Activity Log Mongoose models.
4	Authentication	Register, login, user profile, bcrypt password hashing, JWT generation and verification.
5	Fitness & activity tracking management	Protected workout logging and activity tracking CRUD APIs.
6	Health metric service	Public/Protected daily fitness metrics endpoint with external fitness API integration and local fallback.
7	AI fitness insights service	Protected workout analysis and personalized AI recommendation endpoints with Gemini and fallback logic.
8	API testing	Postman collection covering authentication, fitness logs, health metrics, and AI recommendation endpoints.
9	Execution and demonstration	Run the server, verify endpoints, and record the project demonstration.

2. Suggested Execution Order

- 1 Install dependencies with `npm install`.
- 2 Configure MongoDB and the required environment variables.
- 3 Start the backend using `npm run dev` or `npm start`.
- 4 Verify that MongoDB connects successfully before accepting requests.

- 5 Test user registration and login first to obtain a JWT.
- 6 Use the token for protected profile, workout tracking, and AI insight endpoints.
- 7 Test the daily health and activity metric endpoints.
- 8 Verify fallback behavior if external fitness or Gemini API keys are not available.
- 9 Run the complete Postman collection flow and capture screenshots/video evidence.

3. Project Risks and Mitigation

RISK	PROJECT-SUPPORTED MITIGATION
Missing Fitness / Health Data API key	Use deterministic local mock health and workout data.
Missing/invalid Gemini key or Gemini service error	Use local rule-based fitness summary and workout recommendation generation.
Invalid, expired, or missing JWT token	Reject protected requests with an standardized authorization error response.
Duplicate workout/activity record submission	Validate user ID, activity timestamp, and activity type combination before creation/update.
Database connection failure	Abort server startup gracefully when the initial MongoDB connection fails.

4. Completion Criteria

- Server starts successfully and connects to MongoDB.
- Authentication endpoints work correctly and protected routes reject missing/invalid tokens.
- Workout logging and activity CRUD operations work seamlessly for the authenticated user.
- Health and fitness metrics retrieval returns expected structured fields.
- AI summary and workout recommendation endpoints return generated or fallback results.
- Postman collection can be executed end-to-end to demonstrate the complete API flow.